Inheritance and Polymorphism

Name: Rahul Vedant Pemsing
Student ID: 36305529
Date: 30/09/2025

Learning Journey and Evidence

Prior Knowledge and Experience

From my A-level Computer Science subject, I coded a lot of programs that involved inheritance and polymorphism. I learned how parent classes can define shared attributes and methods, while child classes can extend or override these behaviors to suit specific needs. For example, I created programs where a base class represented a generic shape, and subclasses such as Circle and Rectangle inherited from it while implementing their own methods for calculating area and perimeter.

I also practiced using polymorphism in situations where multiple subclasses shared the same interface but provided different implementations. This gave me an appreciation of how polymorphism allows for flexibility in design, letting one piece of code handle many types of objects seamlessly. These experiences helped me understand the importance of abstraction, code reuse, and writing modular, maintainable programs.

Through these projects, I became comfortable applying OOP principles in real coding tasks, which gave me a solid foundation to approach the inheritance and polymorphism exercises in this unit with confidence.

Additionally from task C2, I also was able to refresh my prior knowledge and learned how inheritance and polymorphism was implemented in C/C++.

Study to Acquire, Refresh, and/or Extend Capability

To study and master this topic I did the following:

30/09/2025 - Read the python guide aboutInheritance and Polymorphism

fopp This Chapter Log-in to Remove Ads! Saving and Logging are Disabled

22.1. Introduction: Class Inheritance

Classes can "inherit" methods and class variables from other classes. We'll see the mechanics of how this works in subsequent sections. First, however, let's motivate why this might be valuable. It turns out that inheritance doesn't let you do anything that you couldn't do without it, but it makes some things a lot more elegant. You will also find it's useful when someone else has defined a class in a module or library, and you just want to override a few things without having to reimplement everything they've done.

Consider our Tamagotchi game. Suppose we wanted to make some different kinds of pets that have the same structure as other pets, but have some different attributes or behave a little differently. For example, suppose that dog pets should show their emotional state a little differently than cats or act differently when they are hungry or when they are asked to fetch something.

You could implement this by making an instance variable for the pet type and dispatching on that instance variable in various methods.

```
from random import randrange

class Pet():
    boredom_decrement = 4
    hunger_decrement = 6
    boredom_threshold = 5
    hunger_threshold = 10
    sounds = ['Mrrp']
    def __init__(self, name = "Kitty", pet_type="dog"):
        self.name = name
        self.hunger = randrange(self.hunger_threshold)
        self.boredom = randrange(self.boredom_threshold)
        self.sounds = self.sounds[:]  # copy the class attribute, so that when we make chan
        self.pet_type = pet_type

    def clock_tick(self):
        self.boredom += 1
        self.hunger += 1

    def mood(self):
        if self.hunger <= self.hunger_threshold and self.boredom <= self.boredom_threshold:
            if self.pet_type == "dog": # if the pet is a dog, it will express its mood in a
                return "happy"
            elif self.pet_type == "cat":
                return "happy, probably"
            else:
                return "HAPPY"
        elif self.hunger > self.hunger_threshold:
            if self.pet_type == "dog": # same for hunger -- dogs and cats will express thei
                return "hungry, arf"
            elif self.pet_type == "cat":
```

From the Python guide, I learned that inheritance allows me to create new classes based on existing ones, which promotes code reuse and makes programs easier to maintain. I also understood how method overriding lets child classes provide their own behavior while still keeping the overall structure from the parent class. Polymorphism showed me how a single function or interface can work with different object types, which simplifies code and increases flexibility.

30/09/2025 - Read the python guide about abstract class

ntrol Flow Functions List String Set Tuple Dictionary Oops Exception Handling Python Programs Python Projects Python Interview Questions Pyth

Abstract Classes in Python

Last Updated : 03 Sep, 2025

In Python, an abstract class is a class that cannot be instantiated on its own and is designed to be a blueprint for other classes. Abstract classes allow us to define methods that must be implemented by subclasses, ensuring a consistent interface while still allowing the subclasses to provide specific implementations.

When to Use Abstract Classes

Abstract classes are useful when you want to:

- Define a common interface for all subclasses (e.g., all animals must have a sound()).
- Enforce implementation of certain methods in child classes.
- Provide shared functionality (concrete methods) while still requiring subclasses to implement specific behavior.

Abstract Base Classes

An Abstract Base Class (ABC) defines methods that must be implemented by its subclasses, ensuring that the subclasses follow a consistent structure. ABCs allow you to define common interfaces that various subclasses can implement while enforcing a level of abstraction.

Python provides the abc module to define ABCs and enforce the implementation of abstract methods in subclasses.

Example: This example shows an abstract class Animal with an abstract method sound() and a concrete subclass Dog that implements it.

```
from abc import ABC, abstractmethod
class Animal(ABC):
    @abstractmethod
    def sound(self):
        pass

class Dog(Animal):
    def sound(self):
        return "Bark"
```

From reading about abstract classes, I learned that they serve as blueprints for other classes and cannot be instantiated directly. Abstract classes can define abstract methods, which act as contracts requiring subclasses to implement them. This ensures consistency across related classes while still allowing each subclass to provide its own unique implementation. I now see how abstract classes improve code organization and enforce a structured design.

30/09/2025 - Coding[Field Guide → Part 2 → 7.Member function → Guided tour → Transaction]
Description: We can explore inheritance and abstract classes by extending our account program to incorporate transactions. In our system each transaction will refer to an account, be for an amount, and will know if it has been performed. When a transaction is performed, it will interact with the account and either deposit or withdraw funds depending on the type of transaction. **My code:**

account.py > DepositTransaction > perform

```python
class BankAccount:
    def __init__(self, name="Unknown", balance=0):
        """Initialize bank account with name and balance"""
        self.name = name
        self._balance = balance

    def print(self):
        """Print account name and balance"""
        print(f"{self.name}: ${self._balance}")

    def deposit(self, amount):
        """Deposit money into account"""
        self._balance += amount
        print(f"${amount} has been deposited")

    def withdraw(self, amount):
        """Withdraw money from account"""
        if self._balance >= amount:
            self._balance -= amount
            print(f"${amount} has been withdraw")
        else:
            print("Not enought money...")

class Transaction:
    def __init__(self, account, amount):
        """Initialize transaction with account and amount"""
        self._performed = False
        self._succeeded = False
        self.amount = amount
        self.account = account

    def perform(self):
        """Perform the transaction"""
        raise NotImplementedError

    def was_performed(self):
        """Check if transaction was performed"""
        return self._performed

    def was_successful(self):
        """Check if transaction was successful"""
        return self._succeeded
```

```python
class WithdrawTransaction(Transaction):
    def perform(self):
        """Perform withdrawal transaction"""
        if self._performed:
            raise Exception("Transaction already performed")

        try:
            self.account.withdraw(self.amount)
            self._succeeded = True
        except Exception as e:
            print(f"Transaction failed: {e}")
            self._succeeded = False
        self._performed = True

class DepositTransaction(Transaction):
    def perform(self):
        """Perform deposit transaction"""
        if self._performed:
            raise Exception("Transaction already performed")

        try:
            self.account.deposit(self.amount)
            self._succeeded = True
        except Exception as e:
            print(f"Transaction failed {e}")
            self._succeeded = False
        self._performed = True
```

```python
if __name__ == "__main__":
    acc = BankAccount("sam", 90125)

    acc.print()

    test = WithdrawTransaction(acc, 1000)
    test.perform()

    if test.was_successful():
        print("Transaction successful!")
    else:
        print("Transaction failed")

    acc.print()

    test = DepositTransaction(acc, 2000)
    test.perform()

    if test.was_successful():
        print("Transaction successful!")
    else:
        print("Transaction failed")

    acc.print()
```

Output:

```
Rahul@VRP_Legion MINGW64 /c/Users/Rahul/Desktop/mycode/m
$ C:/msys64/mingw64/bin/python3.12.exe c:/Users/Rahul/De
sam: $90125
$1000 has been withdraw
Transaction successful!
sam: $89125
$2000 has been deposited
Transaction successful!
sam: $91125
```

By implementing the transaction system, I learned how inheritance can be applied to model real-world problems. I saw how different transaction types (e.g., deposit, withdrawal) could share common behavior from a parent class while defining their own specific logic. I also realized how polymorphism allows me to treat all transactions uniformly when processing them, even though their effects on the

account differ. This helped me understand how object-oriented design makes systems more modular and extendable.

30/09/2025 - Coding[Field Guide → Part 2 → 7. Member function → Activities → Test yourself++ → Dynamic menu]

Description:
If we show a menu of options, then we have a switch or set of if statements to select which action to perform. Adding a new action required us to adjust the output as well as the code to select and run the action.

For example, we may want something like this:

```
1: Say Hello
2: Simple Math
-1: Quit
Option:
```

The user then enters the number of the action they want to perform.

With objects, inheritance, and polymorphism we can now replace this with a list of “action” objects. If each action can output itself to the terminal, then we can assign each a number and use that number to select the action object we tell it to run when the user chooses it.

Mycode:

D_menu.py > choose_and_run

```python
import random
from abc import ABC, abstractmethod

def read_string(prompt):
    """Prompt the user and return a string."""
    return input(prompt)

def read_integer(prompt):
    """Prompt the user and return an integer. Keeps asking until valid."""
    while True:
        try:
            return int(input(prompt))
        except ValueError:
            print("Please enter a integer number.")

class UserAction(ABC):
    """Abstract base class for all user actions."""

    @abstractmethod
    def show(self, option_number):
        pass

    @abstractmethod
    def run(self):
        pass

    def selectable(self):
        return True

class DividerAction(UserAction):
    """A non-selectable divider used for menu formatting."""

    def __init__(self, text="----------------"):
        self.text = text

    def show(self, option_number):
        print(f"   {self.text}")

    def run(self):
        pass  # does nothing

    def selectable(self):
        return False
```

```python
class SayHelloAction(UserAction):
    """Action: Prompt the user's name and greet them."""

    def show(self, option_number):
        print(f"{option_number}: Say Hello")

    def run(self):
        name = read_string("Enter your name: ")
        print(f"Hello, {name}! Nice to meet you.")

class SimpleMathAction(UserAction):
    """Action: Simple math quiz (addition)."""

    def show(self, option_number):
        print(f"{option_number}: Simple Math Test")

    def run(self):
        a = random.randint(1, 10)
        b = random.randint(1, 10)
        answer = read_integer(f"What is {a} + {b}? ")
        if answer == a + b:
            print("Correct!")
        else:
            print(f"Oops, the correct answer was {a + b}")

class GuessDieRollAction(UserAction):
    """Action: Guess the roll of a die."""

    def show(self, option_number):
        print(f"{option_number}: Guess the Die Roll")

    def run(self):
        roll = random.randint(1, 6)
        guess = read_integer("Guess the die roll (1-6): ")
        if guess == roll:
            print("Wow! You guessed it right!")
        else:
            print(f"Sorry, the die rolled {roll}.")
```

```python

class NumberGuessingAction(UserAction):
    """Action: Guess a random number between 1-20."""

    def __init__(self):
        self.high_score = 0

    def show(self, option_number):
        print(f"{option_number}: Number Guessing Game")

    def run(self):
        number = random.randint(1, 20)
        attempts = 0
        print("I'm thinking of a number between 1 and 20!")

        while True:
            attempts += 1
            guess = read_integer(f"Attempt {attempts} - Your guess: ")

            if guess == number:
                print(f"Correct! You got it in {attempts} attempts!")
                if self.high_score == 0 or attempts < self.high_score:
                    self.high_score = attempts
                    print(f"New best score: {self.high_score} attempts!")
                break
            elif guess < number:
                print("Too low!")
            else:
                print("Too high!")
```

```python
def choose_and_run(actions):
    """Show a menu of actions and let the user choose and run them."""
    while True:
        print("\n=== Menu ===")
        option_number = 1
        index_map = []

        for i, action in enumerate(actions):
            if action.selectable():
                action.show(option_number)
                index_map.append(i)
                option_number += 1
            else:
                action.show(-1)

        print("-1: Quit")
        choice = read_integer("Option: ")

        if choice == -1:
            break

        if 1 <= choice <= len(index_map):
            actions[index_map[choice - 1]].run()
        else:
            print("Invalid option, try again.")

def main():
    """Program entry point."""
    actions = [
        SayHelloAction(),
        SimpleMathAction(),
        DividerAction("=== Games ==="),
        GuessDieRollAction(),
        NumberGuessingAction(),
        DividerAction("=== End ===")
    ]

    choose_and_run(actions)
    print("Goodbye!")

if __name__ == "__main__":
    main()
```

Output:

```
Rahul@VRP_Legion MINGW64 /c/Users/R
$ C:/msys64/mingw64/bin/python.exe

=== Menu ===
1: Say Hello
2: Simple Math Test
   === Games ===
3: Guess the Die Roll
4: Number Guessing Game
   === End ===
-1: Quit
Option: 1
Enter your name: sam
Hello, sam! Nice to meet you.

=== Menu ===
1: Say Hello
2: Simple Math Test
   === Games ===
3: Guess the Die Roll
4: Number Guessing Game
   === End ===
-1: Quit
Option: 2
What is 6 + 2? 8
Correct!
```

```
=== Menu ===
1: Say Hello
2: Simple Math Test
   === Games ===
3: Guess the Die Roll
4: Number Guessing Game
   === End ===
-1: Quit
Option: 3
Guess the die roll (1-6): 6
Sorry, the die rolled 1.

=== Menu ===
1: Say Hello
2: Simple Math Test
   === Games ===
3: Guess the Die Roll
4: Number Guessing Game
   === End ===
-1: Quit
Option: 4
I'm thinking of a number between 1 and 20!
Attempt 1 - Your guess: 15
Too low!
Attempt 2 - Your guess: 17
Too low!
Attempt 3 - Your guess: 19
Too high!
Attempt 4 - Your guess: 18
Correct! You got it in 4 attempts!
New best score: 4 attempts!
```

```
=== Menu ===
1: Say Hello
2: Simple Math Test
   === Games ===
3: Guess the Die Roll
4: Number Guessing Game
   === End ===
-1: Quit
Option: -1
Goodbye!
```

From building the dynamic menu, I learned how polymorphism reduces the need for large if or switch statements. Instead of hardcoding conditions, I could create a list of action objects that all shared the same interface. Each object knew how to display itself and execute its own behavior. This made adding or removing actions much easier, showing me how inheritance and polymorphism support scalability and cleaner code.

30/09/2025 - Coding[Field Guide → Part 2 → 7. Member function → Activities → Test yourself++ → Game Action]

Description:
To play around with inheritance and polymorphism, let's build some game mechanics that would allow you to apply effects to a game entity. Using this we can add the ability to control the game entity

My code:

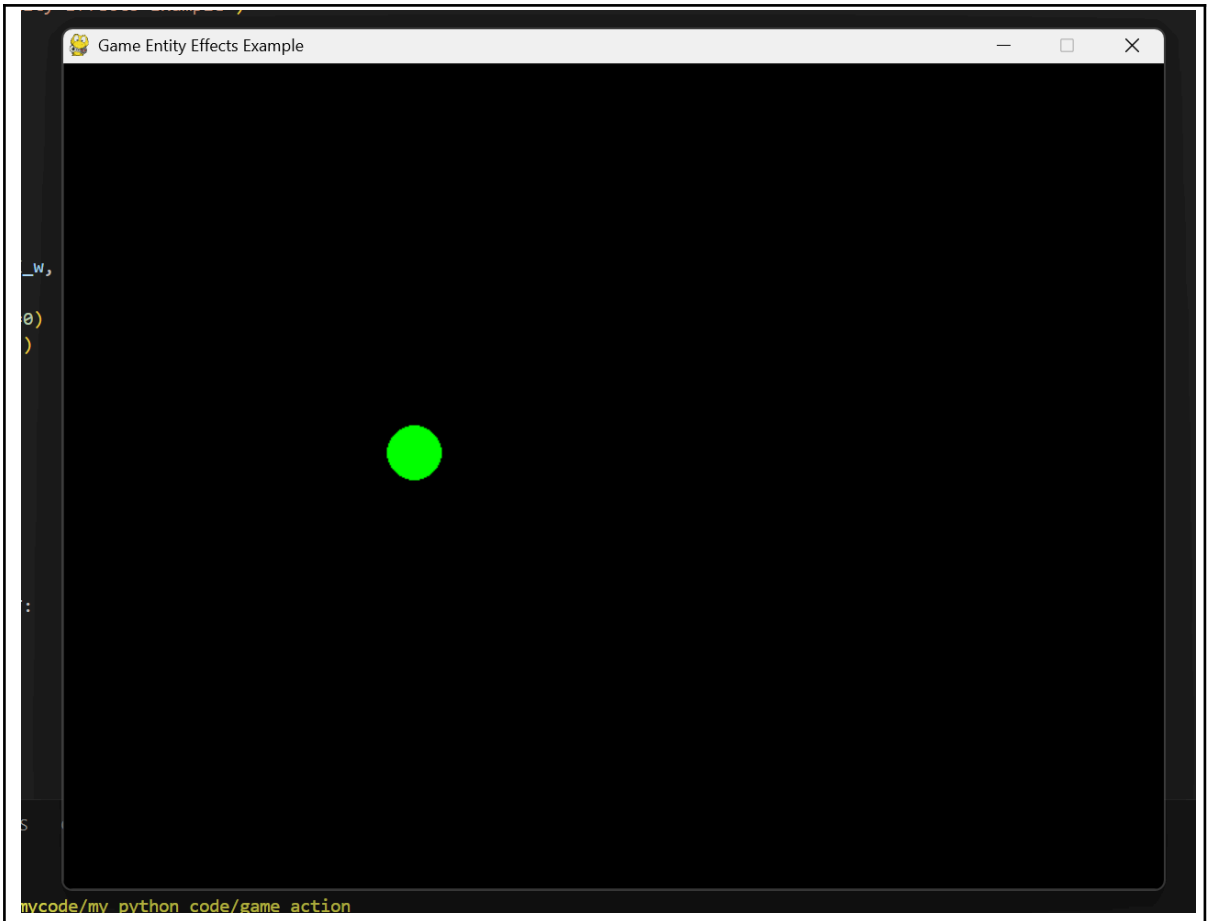
game.py > ...

```python
import pygame
from abc import ABC, abstractmethod


class GameEntity:
    def __init__(self, x=100, y=100, size=20, color=(0, 255, 0)):
        """Initialize game entity with position, size, and color."""
        self.x = x
        self.y = y
        self.size = size
        self.color = color
        self.effects = []  # optional extension: store effects inside the entity

    def draw(self, screen):
        """Draw the entity as a circle on the screen."""
        pygame.draw.circle(screen, self.color, (int(self.x), int(self.y)), self.size)

    def update(self):
        """Perform all effects attached to this entity."""
        for effect in self.effects:
            effect.perform(self)



class EntityEffect(ABC):
    @abstractmethod
    def perform(self, entity: GameEntity):
        """Apply effect to the given entity."""
        pass

```

```python
class KeyboardMoveEffect(EntityEffect):
    def __init__(self, left_key, right_key, up_key, down_key, delta=5):
        """Initialize keyboard movement effect with key bindings and movement speed."""
        self.left_key = left_key
        self.right_key = right_key
        self.up_key = up_key
        self.down_key = down_key
        self.delta = delta

    def perform(self, entity: GameEntity):
        """Move entity based on pressed keys."""
        keys = pygame.key.get_pressed()
        if keys[self.left_key]:
            entity.x -= self.delta
        if keys[self.right_key]:
            entity.x += self.delta
        if keys[self.up_key]:
            entity.y -= self.delta
        if keys[self.down_key (variable) delta: int
            entity.y += self.delta


class WindEffect(EntityEffect):
    def __init__(self, dx=1, dy=0):
        """Initialize wind effect with horizontal and vertical drift values."""
        self.dx = dx
        self.dy = dy

    def perform(self, entity: GameEntity):
        """Apply constant wind drift to entity position."""
        entity.x += self.dx
        entity.y += self.dy



class MouseClickMoveEffect(EntityEffect):
    def perform(self, entity: GameEntity):
        """Move entity to mouse position when left button is clicked."""
        if pygame.mouse.get_pressed()[0]:  # left click
            mouse_x, mouse_y = pygame.mouse.get_pos()
            entity.x, entity.y = mouse_x, mouse_y
```

```python
def main():
    """Main game loop that initializes pygame and runs the game."""
    pygame.init()
    screen = pygame.display.set_mode((800, 600))
    pygame.display.set_caption("Game Entity Effects Example")

    clock = pygame.time.Clock()
    running = True

    # Create player in the center
    player = GameEntity(400, 300)

    # Create effects
    move_effect = KeyboardMoveEffect(
        pygame.K_a, pygame.K_d, pygame.K_w, pygame.K_s, delta=5
    )
    wind_effect = WindEffect(dx=0.2, dy=0)  # gentle push to the right
    mouse_effect = MouseClickMoveEffect()

    # Attach effects to player
    player.effects.append(move_effect)
    player.effects.append(wind_effect)
    player.effects.append(mouse_effect)

    # Game Loop
    while running:
        for event in pygame.event.get():
            if event.type == pygame.QUIT:
                running = False

        # Update player (apply effects)
        player.update()

        # Draw
        screen.fill((0, 0, 0))
        player.draw(screen)
        pygame.display.flip()

        clock.tick(60)

    pygame.quit()


if __name__ == "__main__":
    main()
```

Output:



Through the game action exercise, I learned how polymorphism allows me to design flexible systems where different effects can be applied to game entities without rewriting existing logic. Each action class could implement its own behavior (like moving, attacking, or healing), while still fitting into the same framework. This demonstrated how polymorphism supports creativity and extensibility in programming, especially in game design and simulation tasks.

Additionally I refresh my prior knowledge on how to use the pygame library.

Brief Summary of Concepts

Concept	Key Idea / Concept
Parent Class	A class that provides attributes and methods which can be inherited by other classes. It defines common behavior that child classes can extend or reuse.
Child Class	A class that inherits from a parent class. It can use the parent's attributes and methods, while also adding its own features or overriding existing ones.
Inheritance	A mechanism that allows a child class to acquire the properties and behaviors of a parent class. This promotes code reuse and helps establish a hierarchical relationship between classes.
Polymorphism	The ability of different classes to provide different implementations of the same method name. It allows objects to be treated as instances of their parent type while still exhibiting unique behavior.
Base Class	Another term for a parent class, serving as the foundation from which other classes (derived classes) are created.
Derived Class	Another term for a child class, which inherits from a base class. It may extend or modify the behavior defined in the base class.
Abstract Method	A method declared but not implemented in an abstract class. It acts as a contract, requiring derived classes to provide their own implementation.
Abstract Class	A class that cannot be instantiated directly and may include one or more abstract methods. It serves as a blueprint for subclasses.
Protected	A visibility level where class members are accessible within the class itself and its child classes, but not from outside code. In Python this is typically indicated by a single underscore (_).
Override	When a child class provides a new implementation for a method defined in its parent class. This allows the child class to change or extend the inherited behavior.

Reflection

What is the most important thing you learned from this and why?

The most important thing I learned is how inheritance and polymorphism simplify program design by reducing code duplication and making systems more flexible. These concepts allow me to write more general code while still handling specific cases, which is essential for building scalable and maintainable programs.

How did your understanding of programming concepts help you approach inheritance and polymorphism?

My prior understanding of classes, methods, and encapsulation helped me quickly grasp inheritance and polymorphism. Knowing how data and behavior are encapsulated in classes makes it easier to see how child classes can extend functionality. It also helped me understand why polymorphism is powerful, since it allows different classes to share the same interface while behaving differently.

As you approach the end of the pass content for FIT1045, how prepared do you feel for subsequent units? What strategies do you think will help you succeed in your future programming units?

I feel increasingly confident about taking on more advanced programming units because I now understand the fundamental principles of object-oriented design. To succeed in future units, I plan to:

- Practice consistently by writing code that applies new concepts.
- Review and compare different programming paradigms to broaden my perspective.
- Focus on debugging and testing skills to ensure my code is reliable.
- Collaborate with peers and make use of available resources to strengthen my problem-solving strategies.

References

Generative AI acknowledgement

No AI was used for the code or this document.